Open Access | https://doi.org/10.25080/gcwu7766



SciPy 2026

July 13 - July 19, 2026

Proceedings of the 25th Python in Science Conference
ISSN: 2575-9752

Opening the Black Box: Mechanistic Interpretability for AI Agent Tool Selection Using Sparse Autoencoders

Hannes Hapke[1], **David Cardozo**[2], **Carlos Huisa**[3], and **Triveni Gandhi**[4]

[1]575 Lab, Dataiku Inc., [2]575 Lab, Dataiku Inc., [3]Dataiku Inc., [4]Dataiku Inc.

Abstract

AI agents that autonomously select and run tools are increasingly deployed in business contexts, yet their decision-making processes remain opaque. When an agent chooses to query a database rather than search the web, what internal representations drive that choice? Current approaches rely on prompt engineering, behavioral testing, or post-hoc explanations, none of which reveal the actual computational mechanisms within the model. We present a complete mechanistic interpretability pipeline for understanding agent tool-selection decisions. Our approach extracts hidden-state activations from the subject model at the precise moment of tool commitment (the *decision token*), trains a JumpReLU Sparse Autoencoder (SAE) to decompose these activations into monosemantic features, and applies contrastive analysis to identify which features are associated with specific decision patterns. Crucially, the SAE learns the model's natural feature vocabulary without contrastive supervision; contrastive pairs serve only as post-hoc statistical probes. We validate our feature explanations using token-level fuzzing evaluation adapted from EleutherAI's autointerp methodology, which tests whether labels identify the correct tokens that activate each feature, not just the correct texts. Our pipeline processes diverse agent scenarios across five domains and produces human-readable decision reports explaining the internal factors driving tool selection. Our implementation is open source at https://github.com/dataiku/kiji-inspector.

Keywords mechanistic interpretability, sparse autoencoders, AI agents, tool selection, feature interpretation

Published Jul 13, 2026

Correspondence to
Hannes Hapke
hannes.hapke@dataiku.com

Open Access

Copyright © 2026 Hapke *et al.*. This is an open-access article distributed under the terms of the Creative Commons Attribution 4.0 International license, which enables reusers to distribute, remix, adapt, and build upon the material in any medium or format, so long as attribution is given to the creator.

1. Introduction

The deployment of AI agents capable of autonomous tool selection represents a significant advance in artificial intelligence capabilities. Modern agents can access databases, execute code, search the web, and delegate to sub-agents based on natural language requests. However, this capability comes with a fundamental transparency problem: we cannot inspect the internal decision process that leads an agent to choose one tool over another.

Consider an agent that receives the request "Find information about our company's API rate limits." The agent might choose between searching internal documentation or the public web. The agent's decision to use internal search is the correct one, but what if it instead chose the wrong tool or took an unauthorized action? Without the ability to trace and understand the model's interpretation of both the request and the environment in which it is working, users can not be confident that the agent is taking the right course of action and in turn can not trust it to operate as required. Understanding *why* the model interprets

"our company's" as requiring internal search, rather than treating it as a generic reference, requires inspecting the model's internal representations, not just its outputs.

Current approaches to understanding agent behavior fall into three categories, none of which provide mechanistic insight:

1. **Prompt engineering**: Modifying system prompts and observing behavioral changes reveals correlations but not causal mechanisms.
2. **Behavioral testing**: Probing with test inputs characterizes input-output relationships without explaining intermediate computations.
3. **Chain-of-thought inspection**: Examining generated reasoning text or prompting the model for self-explanations provides a narrative rather than the underlying computation. Prior work [1] demonstrates that such generated rationales can be unfaithful, producing plausible-sounding justifications (confabulations) that do not reflect the internal representations actually driving decisions.

Mechanistic interpretability offers an alternative: directly examining the model's internal representations to identify the computational features that contribute to decisions. Recent work on Sparse Autoencoders (SAEs) has demonstrated that neural network activations can be decomposed into interpretable, monosemantic features that correspond to human-understandable concepts [2], [3]. However, this work has focused primarily on next-token prediction in language models, not on the structured decision-making that characterizes AI agents.

This paper presents a complete pipeline for mechanistic interpretability of agent tool-selection decisions. Our contributions are:

1. **Decision token extraction**: We identify the precise position in the prompt where the model commits to a tool choice and extract activations at this critical moment.
2. **Contrastive pairs as post-hoc probes**: We generate synthetic pairs of semantically similar requests requiring different tools, but use these pairs only for statistical analysis *after* SAE training. The SAE learns the model's natural feature vocabulary unsupervised. Contrastive pairs are used to identify which of these pre-existing features are associated with specific decision differences.
3. **JumpReLU SAE with tanh sparsity**: We employ a JumpReLU activation function that produces exact zeros with learnable per-feature thresholds, trained using a smooth tanh-based sparsity penalty.
4. **Token-level fuzzing evaluation**: We adapt EleutherAI's autointerp methodology [4] to validate whether feature labels correctly identify *which tokens* activate each feature, catching explanations that are "right for the wrong reasons."

The remainder of this paper is organized as follows. Responsible AI Motivations provides the ethical motivation for this work. Related Work reviews related work. Methodology details our methodology. Evaluation: Token-Level Fuzzing describes our evaluation approach. Experimental Setup presents our experimental setup, and Results reports results. Discussion discusses findings and limitations, and Conclusion concludes.

2. Responsible AI Motivations

Transparency of agent decisions is increasingly important as AI regulations are implemented across many geographies and industries. Traditional approaches to AI Governance require the ability to interpret and explain model behavior and agentic systems are subject to the same scrutiny, even if their mechanics are different. Transparency requirements are grounded in the need to verify AI systems are operating as intended, leading to stronger trust in and adoption of the technology. Engendering trust in AI systems requires a frame-

work for responsible AI development; we choose to follow the transparency principle outlined in the RAFT framework [5] as the underpinning and motivation for understanding agent behavior.

The advent of chat-based language models and rapid development in agentic capabilities have highlighted the ongoing need for robust and practical frameworks for AI Governance. Numerous political bodies and standards organizations have developed frameworks to manage the risks associated with AI - though these largely provide high-level ethical principles. By contrast, the RAFT framework is a value-criteria-indicator approach to Responsible AI that covers risks from traditional and generative AI. It is an intentionally lightweight methodology that focuses on what are considered "baseline" requirements for good AI Governance. The RAFT framework is comprised of 4 values - Reliable, Accountable, Fair, and Transparent.

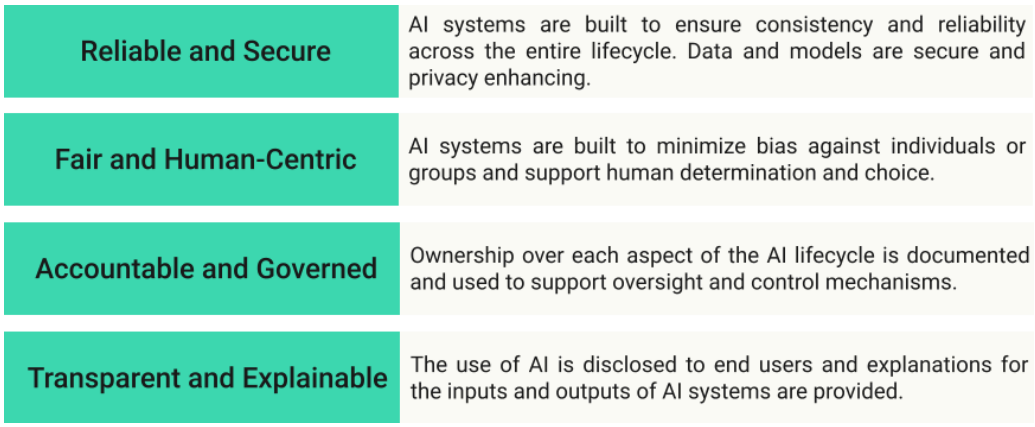


Figure 1. *Core principles of the RAFT framework.*

From these principles organizations form specific criteria and indicators to assess whether their systems align to these values. For example, in the case of Transparency an organization might assign "explainable outputs" as a criterion towards alignment with this principle. Within that criterion will be multiple indicators that can be used as observable measures of progress - such as SHAP values (for predictive modeling) or chain-of-thought reasoning (in the case of generative AI systems). The mechanistic interpretability approach offered in this paper can serve as another indicator towards explainability and overall transparency of agentic systems. Our hope is that by grounding the motivation for this work in a larger principles-based approach we can support the holistic development of responsible AI tooling.

3. Related Work

3.1. Sparse Autoencoders for Interpretability

Sparse autoencoders have emerged as a powerful tool for decomposing neural network activations into interpretable features. T. Bricken *et al.* [2] demonstrated that SAEs trained on language model activations can recover monosemantic features, identifying individual neurons in the SAE that correspond to single, interpretable concepts. This addresses the *superposition hypothesis*, which posits that neural networks represent more features than they have dimensions by encoding multiple concepts in overlapping directions [6].

H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey [3] extended this work with improvements to SAE training, including better initialization strategies and sparsity

penalties. The JumpReLU activation function [7] provides exact sparsity, which improves interpretability by ensuring features are either clearly active or completely inactive.

3.2. Mechanistic Interpretability

Mechanistic interpretability aims to reverse-engineer the algorithms implemented by neural networks [8]. Early work focused on identifying circuits, subgraphs of neurons that implement specific functions [9]. More recent approaches use dictionary learning to find interpretable directions in activation space [10].

3.3. AI Agents and Tool Use

Tool-augmented language models have become a dominant paradigm for AI agents. Toolformer [11] demonstrated that language models can learn to use tools through self-supervised learning. ReAct [12] introduced a framework combining reasoning and acting in language models. Despite rapid progress in agent capabilities, interpretability of agent decisions has received limited attention.

3.4. Automatic Interpretability

Labeling features automatically addresses the scalability challenge of mechanistic interpretability. S. Bills *et al.* [13] used language models to generate explanations for individual neurons. Our token-level fuzzing adapts this approach to validate whether feature labels identify the correct *tokens* that activate each feature.

3.5. Contrastive Methods

Contrastive methods have proven effective for identifying meaningful directions in representation space. Contrastive activation addition [14] demonstrated that differences between activations for contrasting inputs can be used to steer model behavior. Representation engineering [15] extended these ideas to control model outputs. Our approach uses contrastive pairs for post-hoc statistical analysis rather than for training or steering.

4. Methodology

4.1. Pipeline Overview

Our pipeline consists of six sequential steps (see Figure 2):

1. **Contrastive pair generation**: Generate synthetic pairs of user requests sharing the same intent but requiring different tools.
2. **Activation extraction**: Extract hidden-state activations from the subject model at the decision token position.
3. **SAE training**: Train a JumpReLU sparse autoencoder on the raw activations.
4. **Contrastive activation analysis**: Use contrastive pairs as post-hoc probes to identify decision-relevant features.
5. **Feature interpretation**: Label features using an LLM and generate decision reports.
6. **Fuzzing evaluation**: Validate feature labels through token-level A/B testing.

The pipeline uses two large language models in separate memory contexts: a generation model (Qwen3-VL-235B-A22B-Instruct-FP8) for pair generation, feature labeling, and fuzzing judgment; and a subject model (NVIDIA Nemotron-3-Nano-30B-A3B-BF16) for activation extraction. The produced SAE model is built to work in conjunction with the subject model. Crucially, mechanistic interpretability requires full access to model weights and internal hidden states; thus, the subject model must be an open-weights model. Closed-

source API models (which hide internal activations) cannot serve as subject models, though they can optionally replace the auxiliary generation model for offline pair generation and feature labeling.

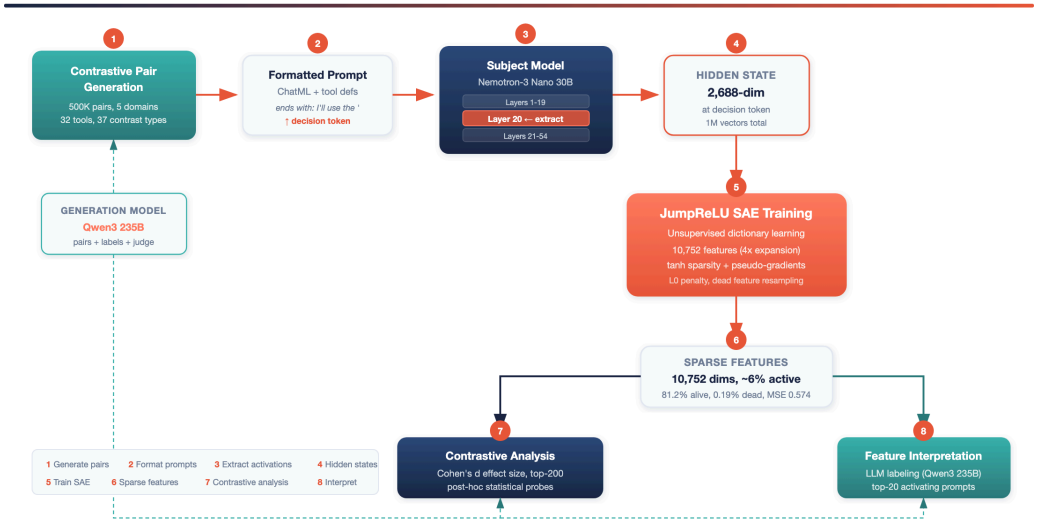


Figure 2. *Overview of the training pipeline. Contrastive pairs are generated and encoded by the subject model. The SAE is trained unsupervised on the extracted activations; contrastive pairs serve only as post-hoc statistical probes for feature analysis and interpretation.*

The study is reproducible from the command line: pair generation is a one-off step, after which a single command runs the remaining stages (Program 1). Flags select individual steps (`--step`), additional layers (`--layers`), or a different subject model (`--subject-model`).

```
python -m kiji_inspector.generate_pairs 1300
python -m kiji_inspector.pipeline
```

Program 1. *Reproducing the study from the command line.*

4.2. Contrastive Pair Generation

Each contrastive pair captures two semantically similar requests that require different tools. The pair structure includes: anchor prompt, anchor tool, contrast prompt, contrast tool, shared intent, and distinguishing signal.

We define scenarios as JSON configurations specifying a domain (e.g., tool selection, investment analysis), available tools, and contrast types. Table 1 summarizes the five scenarios in our system.

Table 1. *Scenario configurations for contrastive pair generation.*

Scenario	Tools	Contrast Types	Example Contrast
tool_selection	8	13	read_vs_write
investment	6	6	risk_vs_return
manufacturing	6	6	quality_vs_speed
supply_chain	6	6	cost_vs_reliability
customer_support	6	6	escalate_vs_resolve

Pairs are generated using an LLM (Qwen3-VL-235B) with a structured prompt template. The generator includes robust JSON parsing with multiple recovery layers: markdown fence stripping, bracket extraction, trailing comma removal, and truncation recovery. Fuzzy key matching handles LLM variations in JSON field names. Program 2 shows three example pairs from different scenarios.

```
Example Pair 1: customer_support (self_service_vs_agent_assist)
Shared Intent: resolve password reset issue
ANCHOR [knowledge_base]: How do I reset my password if I forgot it?
CONTRAST [ticket_lookup]: I tried resetting my password 3 times
but the email never arrives.

Example Pair 2: investment (growth_vs_value)
Shared Intent: Evaluate energy sector stocks for potential investment
ANCHOR [financial_analysis]: Which energy companies are investing
heavily in renewable expansion?
CONTRAST [market_data_lookup]: Which energy stocks are currently
trading below their book value?

Example Pair 3: tool_selection (read_vs_write)
Shared Intent: Check latest product version
ANCHOR [file_read]: What is the latest version of Product X?
CONTRAST [file_write]: Set the latest version of Product X to v3.2.1
```

Program 2. *Example contrastive pairs from three scenarios. Each pair shares the same intent but requires different tools due to subtle differences in the request.*

4.3. Decision Token Extraction

Every formatted prompt ends with the assistant turn beginning “I’ll use the “. The hidden state at this final token, the **decision token**, captures the model’s internal representation at the moment it commits to a tool name.

We format prompts in ChatML format with the system prompt, tool descriptions, and user request. Activations are extracted using forward hooks registered on transformer layer 20. For the Nemotron-3-Nano-30B model (a Mixture-of-Experts architecture with 30B total parameters, 3B active per token), the hidden dimension is 2,688. Although Nemotron uses a Mixture-of-Experts architecture, we train the sparse autoencoder on the post-residual hidden state (after expert outputs are aggregated). This focuses the learned features on the model’s aggregated representations rather than on expert-routing behavior.

Batched extraction uses left-padding to align the decision token across variable-length prompts. Activations are cast to float16 and saved as NumPy shards for efficient loading during SAE training.

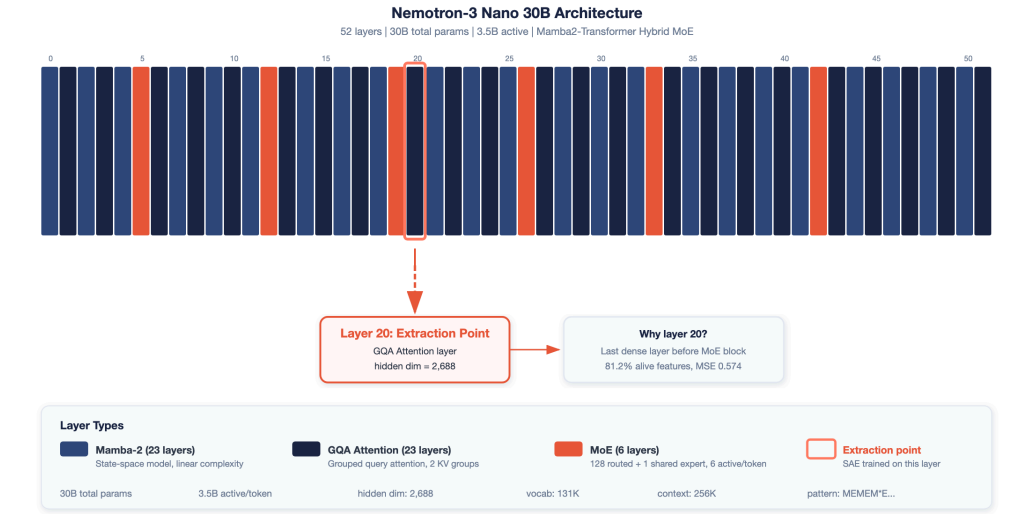


Figure 3. *Architecture of the Nemotron-3-Nano-30B subject model showing all 52 layers. The model interleaves Mamba2 (23 layers), GQA Attention (23 layers), and Mixture-of-Experts (6 layers). Activations are extracted at layer 20, a GQA Attention layer in the mid-network, before the MoE layers that introduce sparse expert routing.*

4.4. JumpReLU SAE Architecture

Our sparse autoencoder uses the JumpReLU activation function [7], which produces exact zeros through learnable per-feature thresholds $\boldsymbol{\theta} \in \mathbb{R}_+^M$.

Encoder. Given input activation $\boldsymbol{x} \in \mathbb{R}^n$, we compute pre-activations $\boldsymbol{\pi}(\boldsymbol{x}) \in \mathbb{R}^M$:

$$\boldsymbol{\pi}(\boldsymbol{x}) = W_{\text{enc}}(\boldsymbol{x} - \boldsymbol{b}_{\text{dec}}) + \boldsymbol{b}_{\text{enc}} \tag{1}$$

The feature activations $\boldsymbol{f}(\boldsymbol{x}) \in \mathbb{R}^M$ are then:

$$f_i(\boldsymbol{x}) = \text{JumpReLU}_{\theta_i}(\pi_i(\boldsymbol{x})) = \pi_i(\boldsymbol{x}) \cdot H(\pi_i(\boldsymbol{x}) - \theta_i) \tag{2}$$

where H is the Heaviside step function and $M = 4 \times n$ is the dictionary size.

Decoder. The reconstruction $\hat{\boldsymbol{x}} \in \mathbb{R}^n$ is:

$$\hat{\boldsymbol{x}}(\boldsymbol{f}) = W_{\text{dec}}\, \boldsymbol{f} + \boldsymbol{b}_{\text{dec}} \tag{3}$$

where the columns of W_{dec} are dictionary directions $\boldsymbol{d}_i$, which are normalized to unit norm after each training step. The decoder bias $\boldsymbol{b}_{\text{dec}}$ is shared: it is subtracted before encoding and added after decoding.

Loss function. Following S. Rajamanoharan *et al.* [7], the loss combines reconstruction with an L0 sparsity penalty:

$$\mathcal{L}(\boldsymbol{x}) = \mid \boldsymbol{x} - \hat{\boldsymbol{x}}(\boldsymbol{f}(\boldsymbol{x})) \mid_2^2 + \lambda \mid \boldsymbol{f}(\boldsymbol{x}) \mid_0 \tag{4}$$

The L0 norm counts active features via the Heaviside function:

$$\mid \boldsymbol{f}(\boldsymbol{x}) \mid_0 = \sum_{i=1}^{M} H(\pi_i(\boldsymbol{x}) - \theta_i) \tag{5}$$

Since H is non-differentiable, we use a smooth tanh approximation in practice:

$$\hat{\mathcal{L}}_{\text{sparse}}(\boldsymbol{x}) = \sum_{i=1}^{M} \text{ReLU}\left(\tanh\left(\frac{\pi_i(\boldsymbol{x}) - \theta_i}{\varepsilon}\right)\right) \tag{6}$$

Pseudo-gradients. Because the Heaviside function has zero gradient almost everywhere, S. Rajamanoharan *et al.* [7] define pseudo-derivatives using a kernel density estimator with bandwidth ε and rectangular kernel K. For the JumpReLU activation:

$$\frac{\tilde{\partial}}{\partial \theta_i} \text{JumpReLU}_{\theta_i}(z) = -\frac{\theta_i}{\varepsilon} K\left(\frac{z - \theta_i}{\varepsilon}\right) \tag{7}$$

For the Heaviside step function in the L0 penalty:

$$\frac{\tilde{\partial}}{\partial \theta_i} H(z - \theta_i) = -\frac{1}{\varepsilon} K\left(\frac{z - \theta_i}{\varepsilon}\right) \tag{8}$$

where $K(u) = \frac{1}{2} 1\{|\, u \,| \leq 1\}$ is the rectangular kernel. Gradients with respect to pre-activations use a standard straight-through estimator: $\tilde{\partial} f_i / \partial \pi_i = 1\{\pi_i > \theta_i\}$.

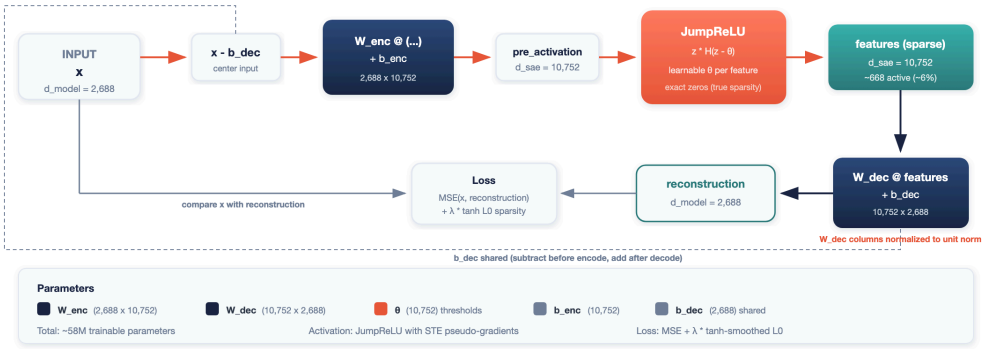


Figure 4. *JumpReLU Sparse Autoencoder architecture. The input activation x is centered by subtracting b_{dec}, projected to a 10,752-dimensional latent space via W_{enc}, and sparsified by the JumpReLU activation with learnable thresholds θ. The decoder reconstructs via $W_{\text{dec}} + b_{\text{dec}}$ (shared bias). The loss combines reconstruction MSE with a smooth tanh-based L0 sparsity penalty.*

Training details. Table 2 summarizes key hyperparameters. Training uses cosine learning rate decay with linear warmup, sparsity coefficient warmup, and dead feature resampling. Decoder columns are normalized to unit norm after each step.

Table 2. *SAE training hyperparameters.*

Parameter	Value	Description
M	10,752	Dictionary size ($4 \times n$)
Batch size	8,192	Activations per training step
Learning rate	3×10^{-4}	Peak learning rate
λ	5×10^{-3}	Sparsity coefficient
ε	0.001	JumpReLU bandwidth
Warmup	5% of steps	LR warmup fraction
Sparsity warmup	10% of steps	Sparsity coefficient warmup
Resample interval	20% of steps	Dead feature resampling

4.5. Contrastive Feature Analysis

After SAE training, we identify decision-relevant features using contrastive pairs as post-hoc statistical probes. For each pair, we extract fresh activations and encode through the trained SAE, then compute per-feature statistics.

Cohen's d effect size. For each feature j across pairs of a contrast type:

$$d_j = \frac{|\ \bar{f}_j^{(a)} - \bar{f}_j^{(c)}\ |}{s_{\text{pooled}}} \tag{9}$$

where $\bar{f}_j^{(a)}$ and $\bar{f}_j^{(c)}$ are mean activations for anchor and contrast prompts, and s_{pooled} is the pooled standard deviation.

Filtering. We apply two filters before ranking: effect size $d_j \geq 0.3$ (small-to-medium effect) and minimum activation $\max\Big(|\ \bar{f}_j^{(a)}\ |, |\ \bar{f}_j^{(c)}\ |\Big) > 0.01$. The top-K features (default: 200) per contrast type are selected for interpretation.

4.6. Feature Interpretation

For each decision-relevant feature, we collect the top-20 highest-activating prompts and bottom-10 near-zero prompts from the activation dataset. An LLM (Qwen3-VL-235B) receives these examples and generates a short label (3-8 words), a one-sentence description, and a confidence rating (high/medium/low).

5. Evaluation: Token-Level Fuzzing

5.1. Motivation

Feature labels may be "right for the wrong reasons", correctly predicting which prompts activate a feature but for an incorrect conceptual reason. Token-level evaluation addresses this by testing whether labels identify the specific *tokens* that drive feature activation.

5.2. Fuzzing Methodology

Per-token extraction. Unlike decision-token extraction (single position), fuzzing extracts activations for every token in the prompt, yielding a $(seq_len, d_{\text{model}})$ matrix per prompt.

User request span detection. We locate the user request within the formatted prompt using ChatML structural markers. We highlight only user-request tokens, since highlighting system-prompt or tool-description tokens would be uninformative.

Token highlighting. For each (prompt, feature) pair: (1) encode per-token activations through the SAE, (2) extract the feature column for the user request span, (3) highlight top-K tokens (adaptive K: at most 1/3 of user tokens), and (4) mark with double angle brackets.

A/B comparison. Token-level examples pair a highlighted top-activating prompt with a highlighted bottom-activating prompt. The A/B order is randomized to prevent position bias. An LLM judge determines which highlighted text better matches the feature label.

5.3. Metrics

Combined score. Token-level accuracy is weighted more heavily:

$$\text{combined} = 0.7 \cdot \text{acc}_{\text{token}} + 0.3 \cdot \text{acc}_{\text{prompt}} \tag{10}$$

Token-level receives higher weight because it tests the actual mechanism (which tokens trigger the feature), not just text-level correlation.

Quality tiers. Features are classified by combined score: Excellent (> 0.8), Good ($0.6 - 0.8$), Poor (< 0.6). Random guessing yields 50% accuracy (binary A/B choice).

6. Experimental Setup

Hardware. Experiments were conducted on a GB200 NVL4 instance with four B200 GPUs (768 GB total GPU VRAM).

Models. We use two models:

1. **Generation/Labeling/Judge**: Qwen/Qwen3-VL-235B-A22B-Instruct-FP8 via vLLM with tensor and expert parallelism.
2. **Subject model**: nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16 via HuggingFace Transformers with automatic device mapping.

Scenarios. We evaluate across five domains with a total of 32 tools and 37 contrast types. The tool_selection scenario serves as the primary benchmark with 8 general-purpose tools and 13 contrast types.

Dataset scale. The pipeline is designed for large-scale operation. For benchmarking, we generate 500,000 contrastive pairs, yielding 1,000,000 activation vectors.

Layer selection. To justify our choice of extraction layer, we sweep across layers $\{8, 16, 20, 32, 44\}$ of the 54-layer Nemotron architecture, running steps 2–4 (activation extraction, SAE training, contrastive feature identification) at each depth using a stratified subsample of 5,000 pairs per contrast type.

7. Results

7.1. Layer Sweep

Table 3 reports SAE quality metrics across five extraction depths. Early layers (8, 16) achieve low reconstruction error but represent pre-decision processing. Mid-network layers (20) balance reconstruction fidelity with decision-relevant representations: layer 20 achieves the highest alive feature percentage (81.2%) and lowest dead feature rate (0.19%) while maintaining moderate L0 sparsity. Deep layers (32, 44) exhibit dramatically higher reconstruction MSE (>500× that of layer 20), suggesting the SAE struggles to faithfully capture these representations—likely because layers 32+ in the Nemotron architecture are Mixture-of-Experts (MoE) layers with sparse expert routing, producing activations that are harder to decompose with a single autoencoder.

Table 3. *Layer sweep results. SAE quality metrics across transformer depths ($M = 16,384$, 5K pairs per contrast type, steps 2–4 only). Layer 20 (bold) is selected for all subsequent experiments.*

Layer	Alive %	Dead %	L0 (mean ± SEM)	Recon. MSE	Features
8	85.3	0.10	540 ± 20	0.031	192
16	63.2	0.85	539 ± 13	0.333	200
20	**81.2**	**0.19**	$\mathbf{668 \pm 26}$	**0.574**	**200**
32	69.5	2.00	$2,231 \pm 11$	1{,}524	200
44	72.0	1.01	$2,060 \pm 12$	508	200

We select layer 20 based on three criteria: (1) highest alive feature utilization, (2) lowest dead feature rate, and (3) reconstruction MSE below 1.0, indicating the SAE faithfully captures the residual stream. While layer 8 has lower MSE, its representations precede the

model's tool-selection reasoning; layers 32 and 44 have reconstruction errors three orders of magnitude higher, making their SAE decompositions unreliable.

7.2. Feature Health Analysis

Post-training analysis of the layer-20 SAE on the full dataset (1M activation vectors) reveals the distribution of feature activity. Table 4 summarizes the metrics.

Table 4. *SAE feature health metrics (layer 20, full dataset).*

Metric	Value
Total features (M)	10,752
Alive features (>0.1% firing rate)	81.2% [80.6, 81.8]
Dead features (0% firing rate)	0.19% [0.13, 0.27]
L0 (mean active features per input)	668 ± 26 (SEM)
Reconstruction MSE	0.574

The 81.2% alive feature rate indicates that the majority of SAE capacity is utilized, while the L0 statistic confirms sparse encoding (668 active features out of 10,752 total, i.e., 6.2% activation density).

7.3. Baseline Comparisons

To contextualize the SAE's contribution, we evaluate two baselines on the same layer-20 activations (841,282 vectors across 32 tools). Table 5 reports the results.

Table 5. *Baseline comparison on raw layer-20 activations (841K vectors, 32 tools). Linear probe uses 5-fold GroupKFold cross-validation; PCA+k-means uses 50 components and 32 clusters.*

Method	Metric	Value
Linear probe	Accuracy	0.796 ± 0.001 (SEM)
Linear probe	Macro F1	0.765 ± 0.001 (SEM)
PCA + k-means	Purity	0.175
PCA + k-means	NMI	0.225
PCA + k-means	ARI	0.068

A logistic regression on raw activations predicts the correct tool with 79.6% accuracy across 32 classes, confirming that tool identity is linearly encoded at layer 20. However, this supervised baseline provides no interpretability, it cannot explain *why* a particular tool was chosen. Unsupervised PCA+k-means fails to recover tool structure (NMI $= 0.225$, ARI $= 0.068$), indicating that tool-relevant signal is not the dominant source of variance: the first 50 principal components capture only 48% of total variance, and the resulting clusters do not align with tool boundaries. The SAE bridges this gap by decomposing the same activations into monosemantic features that are both interpretable (91.2% fuzzing score) and, for select contrast types, causally linked to decisions (Section 20).

7.4. Contrastive Feature Discovery

Contrastive analysis identifies features that systematically differ between anchor and contrast prompts. Top-200 features are selected per contrast type after filtering. A deduplication ratio of 0.7–0.8 indicates moderate feature sharing across contrast types. Top features exhibit Cohen's d > 0.8 (large effect).

7.5. Fuzzing Evaluation

Table 6 summarizes the fuzzing evaluation results across 402 features and 4,422 examples.

Table 6. *Fuzzing evaluation results (402 features, 4,422 examples).*

Metric	Value
Features evaluated	402
Mean combined score	0.912 ± 0.008 (SEM), $p < 10^{-4}$ vs 0.5
Token-level accuracy	0.906 ± 0.007 (SEM), $p < 10^{-4}$ vs 0.5
Excellent (> 0.8)	84.3% (339 features)
Good (0.6–0.8)	9.4% (38 features)
Poor (< 0.6)	6.2% (25 features)

Both the combined score and token-level accuracy are significantly above the 50% random baseline ($p < 10^{-4}$, one-sample t-test), with 84.3% of features achieving excellent quality. Notably, a Kruskal-Wallis test found no significant difference across confidence tiers ($p = 1.0$), indicating the labeling LLM's self-assessed confidence is not predictive of actual fuzzing accuracy in this evaluation.

7.6. Feature Ablation

To test whether contrastive features are *potentially causally* involved in tool-selection decisions (not merely correlated), we perform a feature ablation experiment. For each contrast type, we intercept the residual stream at layer 20, encode through the SAE, zero out the top-10 contrastive features, decode back, and measure whether the model's tool prediction changes. We include two controls: (1) ablating 10 random non-contrastive features, and (2) a *reconstruction-only* baseline that encodes and decodes through the SAE with no features zeroed, measuring distortion from the SAE round-trip alone. Table 7 reports the results for the six contrast types with the highest statistical significance.

Table 7. *Feature ablation results (top-10 features ablated, 100 pairs sampled per contrast type). Reconstruction baseline measures SAE round-trip distortion with no features zeroed. Significance via Fisher's exact test (contrastive vs. random).*

Contrast type	n	Contr.	Directed	Rand./Recon.	p
fundamental vs. technical	89	10.1%	**9.0%**	0.0%	**0.002**
single vs. multi-tool	41	17.1%	2.4%	2.4%	**0.029**
query vs. mutate	9	77.8%	0.0%	33.3%	0.077
single-stock vs. portfolio	65	10.8%	0.0%	6.2%	0.265
root-cause vs. symptom-fix	15	26.7%	13.3%	13.3%	0.326
shallow vs. deep	46	8.7%	0.0%	4.3%	0.338
Aggregate (23 types)		*16.1%*	*2.3%*	*13.0%*	

The strongest evidence of causal involvement emerges for *fundamental vs. technical analysis* ($p = 0.002$): ablating contrastive features flips 10.1% of predictions (9.0% directed toward the contrast tool), while both random ablation and reconstruction-only produce zero flips. This demonstrates that these specific features are causally necessary for the distinction. A second significant result appears for *single vs. multi-tool* ($p = 0.029$), where contrastive ablation produces a 17.1% flip rate against a 2.4% baseline.

Critically, the reconstruction-only baseline reveals that the random ablation flip rate equals the SAE round-trip distortion rate across all 23 contrast types, zeroing 10 random features

from ~ 668 active features adds no disruption beyond what the encode-decode cycle itself introduces. This validates the experimental design: the SAE reconstruction is sufficiently faithful that ablation effects can be attributed to the specific features removed rather than general signal degradation. For contrast types where no ablation effect is observed (e.g., *preventive vs. reactive maintenance*, $n = 95$, 0% across all conditions), the model's tool-selection decision is robust to removing any 10 features, suggesting these decisions rely on distributed representations rather than sparse feature circuits.

Scaling Analysis via Continuous Conditional Average Treatment Effects (CATE)

While discrete token-flip metrics provide clear evidence of macroscopic causal shifts, they fail to capture sub-threshold continuous perturbations within the model's logit distribution. To map these subtler causal dynamics and evaluate the scaling boundaries of the sparse autoencoder (SAE) latent space, we expand the baseline framework across two dimensions: evaluation scale and intervention breadth. We scale the evaluation pool up to $N = 500$ prompt pairs and systematically ablate expanding feature horizons—specifically targeting the top-10, top-20, and top-30 most active contrastive features.

To evaluate these continuous shifts without assuming normality, we replace the categorical Fisher's exact test with the non-parametric Wilcoxon signed-rank test. We calculate the **Continuous Conditional Average Treatment Effect (CATE)**, defined as the average shift in log-probability (Δ Prob) assigned to the target tool output following ablation. Table 8 displays the multi-feature scaling results for key representative contrast types.

Table 8. *Continuous causal ablation scaling results ($N = 500$ prompt pairs). CATE represents the mean continuous probability drop (Δ Prob) for target tool selection. Significance (p) is computed via the Wilcoxon signed-rank test across expanding feature intervention sizes.*

Contrast Type	N Valid	CATE (Top-10)	CATE (Top-20)	CATE (Top-30)	Stable Significance
single vs. batch	175	0.0861	0.0956	**0.1170**	Yes ($p < 0.001$)
specific vs. broad	136	0.0401	0.0672	**0.0842**	Yes ($p < 0.001$)
query vs. mutate	53	0.0229	0.0321	0.0147	Yes ($p < 0.002$)
authoritative vs. general	463	0.0019	0.0033	0.0046	Yes ($p < 0.001$)
local vs. remote	399	**0.0000**	0.0000	0.0000	No (Top-10 only)
read vs. write	112	0.0018	0.0069	0.0041	No (Null Effect)

Our scaling analysis reveals two distinct structural architectures governing how abstract tool-selection criteria are mapped inside the model's sparse latent geometry:

1. **Distributed Latent Escalation:** Core conceptual dualities such as *single_vs_batch* and *specific_vs_broad* exhibit a striking monotonic scaling effect. As the ablation window widens from 10 to 30 features, the causal penalty (CATE) scales upward near-linearly—climbing from 0.0861 to 0.1170 for *single_vs_batch*, and more than doubling from 0.0401 to 0.0842 for *specific_vs_broad*. This behavior provides robust evidence of highly distributed directional circuits; intervening on a broader sparse basis continually intensifies the causal suppression without hitting an early saturation threshold.
2. **Sharp Latent Localization:** Conversely, dimensions like *local_vs_remote* expose a highly localized topological boundary. Under a focused top-10 feature intervention, the probability shift is highly statistically significant ($p < 0.001$). However, when expanding the intervention boundary to 20 or 30 features, the statistical significance completely vanishes, and the CATE stabilizes at ≈ 0.0000. This indicates

that the causal steering vectors for these properties are confined to an incredibly narrow, sparse feature band; broadening the ablation window introduces orthogonal background features and localized noise that completely washes out the downstream causal signal.

Finally, cross-checking these interventions across varying prompt boundaries ($250, 400,$ and 500 instances) confirms strict dataset scale invariance. The structural preservation of our significance clusters and CATE trajectories across all 9 experimental configurations (3 dataset sizes $\times 3$ feature horizons) demonstrates that these latent geometries represent stable internal mechanics of the subject network rather than artifacts of prompt distribution sizing.

8. Discussion

8.1. Key Findings

SAEs discover interpretable decision factors. The trained SAE decomposes tool-selection activations into features that correspond to human-understandable concepts. Features like "internal knowledge retrieval," "data modification intent," and "query complexity" emerge without explicit supervision. Token-level fuzzing confirms that 84.3% of these features achieve excellent quality (> 0.8 combined score), with the aggregate score of 0.912 ± 0.008 significantly exceeding the random baseline ($p < 10^{-4}$).

Selective causal evidence via ablation. Feature ablation demonstrates that contrastive features are causally involved in specific tool-selection decisions: for *fundamental vs. technical analysis*, zeroing the top-10 contrastive features flips 10.1% of predictions (9.0% directed) while random ablation and reconstruction-only produce zero flips ($p = 0.002$). However, this causal link is contrast-type-dependent, for 9 of 23 tested types, neither contrastive nor random ablation produces any flips, indicating that these decisions are distributed across many features rather than concentrated in a sparse circuit. The reconstruction-only baseline proved essential for interpreting these results: random ablation flip rates match the SAE round-trip distortion exactly, confirming that observed effects are attributable to specific feature removal.

Token-level validation catches spurious labels. Prompt-level accuracy alone would overestimate label quality. Some features achieve high prompt-level accuracy but poor token-level accuracy, indicating labels that capture text-level correlations without identifying the actual triggering tokens.

Multi-domain applicability. The same pipeline architecture applies across diverse scenarios (tool selection, investment, manufacturing, supply chain, customer support), demonstrating generalizability.

Practical application. At inference time, the trained SAE integrates directly into the agent's forward pass (Figure 5). A user prompt is encoded by the subject model; at layer 20 the hidden state is intercepted and projected through the SAE to obtain a sparse feature vector. The most active dimensions are mapped to pre-computed feature labels, producing a human-readable decision report alongside the agent's tool selection.

To make this extraction step practical at serving time, the repository ships a patch for vLLM that adds an `--extract-activation-layers` option: it captures prompt activations for the selected layers and returns them in the OpenAI-compatible chat and completion responses (supporting the Nemotron and Gemma-3 architectures). Once applied, the subject model can be served with hidden-state extraction enabled at the decision layer (Program 3):

```
vllm serve nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16 \
    --extract-activation-layers 20
```

Program 3. *Serving the subject model with layer-20 hidden-state extraction enabled.*

We release trained SAEs for several subject models on the Hugging Face Hub (Table 9). Given a subject model, `SAE.from_pretrained` resolves the matching SAE repository from kiji-inspector's built-in registry and loads it together with its feature labels in a single call (Program 4):

```
from kiji_inspector import SAE

# The SAE repository is resolved from kiji-inspector's built-in registry
sae, feature_descriptions = SAE.from_pretrained(
    base_model="nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16",
    layer=20,
)
```

Program 4. *Loading the trained SAE and its feature labels for the paper's subject model.*

Table 9. *Subject models with trained SAEs in the kiji-inspector registry.*

Subject model	SAE repository (Hugging Face)	Group
`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-BF16`	`575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-BF16`	NVIDIA
`nvidia/NVIDIA-Nemotron-3-Nano-30B-A3B-FP8`	`575-lab/kiji-inspector-NVIDIA-Nemotron-3-Nano-30B-A3B-FP8`	NVIDIA
`google/gemma-3-27b-it`	`575-lab/kiji-inspector-google-gemma-3-27b-it`	Experimental
`google/gemma-4-E4B-it`	`575-lab/kiji-inspector-google-gemma-4-E4B-it`	Experimental
`google/gemma-4-31B-it`	`575-lab/kiji-inspector-google-gemma-4-31B-it`	Experimental
`google/gemma-4-26B-A4B-it`	`575-lab/kiji-inspector-google-gemma-4-26B-A4B-it`	Experimental

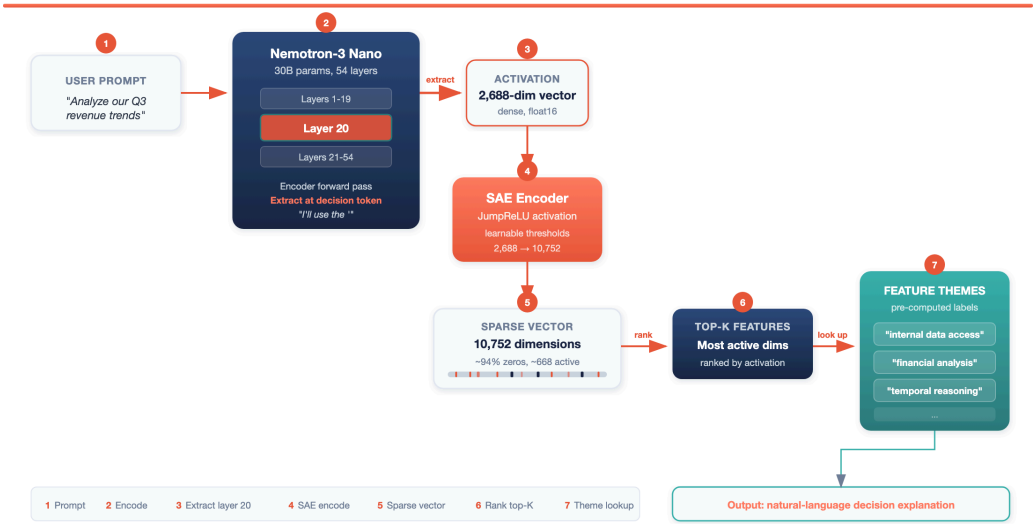


Figure 5. *Inference pipeline for real-time decision explainability. A user prompt is encoded by the Nemotron subject model; the hidden state at layer 20 is intercepted and projected through the trained SAE*

into a sparse 10,752-dimensional feature vector. The top-K active dimensions are mapped to pre-computed feature labels to produce a human-readable decision report.

To demonstrate end-to-end utility, we built an interactive demo application (Figure 6) that runs the full pipeline on user-specified prompts. The interface surfaces SAE-derived explanations alongside the agent's output, translating internal feature activations into natural-language rationales. In the home-improvement demo shown, the system explains which features (e.g., Old Machine Troubleshooting) drove the agent's decision to select the "Repair Manual Lookup", providing actionable transparency for domain experts.

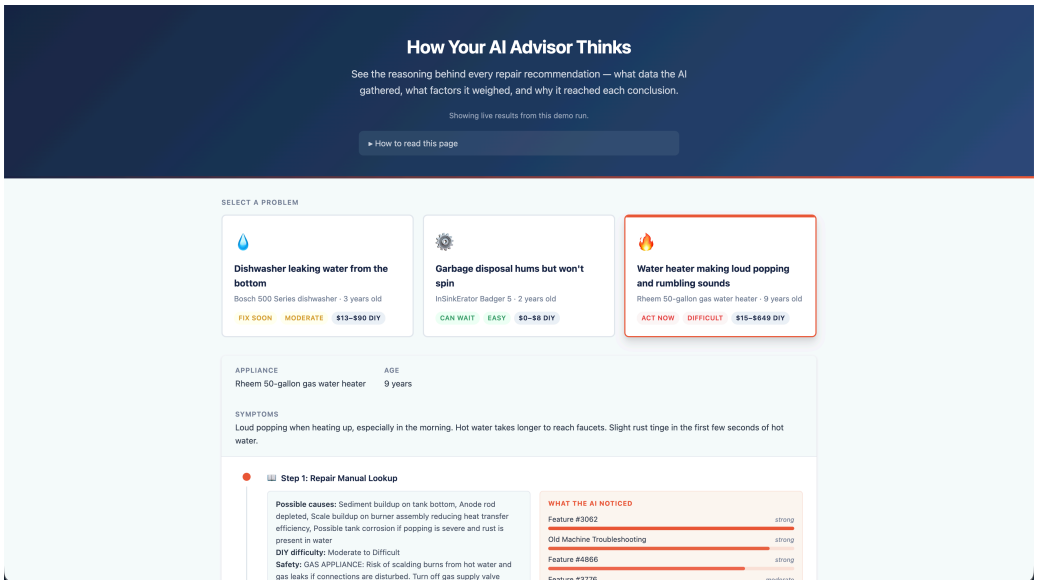


Figure 6. *Interactive demo application showing SAE-powered explainability for a home improvement agent. The demo shows dominant features for every selected tool alongside a natural-language explanation derived from the activated SAE features.*

A runnable, open-source quickstart is provided as supplementary material (Quickstart: Inspecting Agent Tool-Selection Features). Rather than reproducing the results above, it applies the pipeline to a smaller home-repair dataset using a JumpReLU SAE trained on that scenario (distinct from the SAE reported in this paper). In a handful of cells it captures a decision-token activation from the subject model, loads that pretrained SAE with `SAE.from_pretrained`, and reports the top features driving tool selection; it requires only `transformers` and `kiji-inspector` and runs on CPU. It can also be executed directly in Google Colab:



8.2. Limitations

Compute requirements. The pipeline requires substantial GPU resources: a 235B parameter model for generation/labeling/judging and a 30B parameter subject model for activation extraction.

Label quality depends on labeling LLM. Feature interpretation relies on the labeling model's ability to identify patterns in max-activating examples. Labeling errors propagate to the decision report.

Coverage of decision factors. Contrastive pairs are generated synthetically and may not cover all factors that influence tool selection in real deployments.

Ablation scope. Our ablation experiment zeroes the top-10 contrastive features out of ~ 668 active features per input. For contrast types where this produces no effect, the decision may depend on a larger set of features acting in concert. Ablating more features risks general disruption that confounds causal attribution, while ablating fewer reduces statistical power. The optimal ablation set size remains an open question.

8.3. Future Work

Higher-fidelity SAEs for causal intervention. Our ablation results show that the SAE reconstruction-only baseline introduces 13% flip rate on average. Training SAEs with lower reconstruction error, via larger dictionaries, improved architectures (e.g., Gated SAEs [16]), or per-layer optimization, would enable cleaner causal experiments with larger ablation sets.

Multi-layer and circuit-level analysis. The current pipeline operates on a single layer. Extending to multi-layer analysis could reveal how tool-selection features emerge and transform across the network, and whether the MoE layers (32+) that proved difficult for single-layer SAEs could be better captured with expert-specific decompositions.

Cross-model transfer. Testing whether SAE features and their causal roles transfer across model families (e.g., from Nemotron to Llama or Qwen) would establish whether tool-selection circuits are architecture-dependent or reflect shared computational strategies.

Real-time agent monitoring. Integrating the pipeline into production agent systems could provide real-time decision explanations and anomaly detection, flagging when an agent's internal feature activations deviate from expected patterns for a given tool choice.

9. Conclusion

We have presented a complete mechanistic interpretability pipeline for understanding AI agent tool-selection decisions. By extracting activations at the precise moment of tool commitment, training sparse autoencoders to decompose these activations into interpretable features, and validating explanations through token-level fuzzing, we provide insight into the internal factors driving agent decisions.

Our key methodological contributions include: (1) decision token extraction for capturing the moment of tool commitment; (2) contrastive pairs as post-hoc probes rather than training signals; (3) JumpReLU SAE with tanh sparsity for exact zeros and smooth gradients; and (4) token-level fuzzing evaluation that catches explanations which are "right for the wrong reasons."

As AI agents are deployed in increasingly high-stakes contexts, understanding *how* they make decisions, not just *what* decisions they make, becomes critical for safety, debugging, and trust.

10. AI Attribution

In keeping with the SciPy generative AI disclosure policy, we report the use of generative AI tools in the preparation of this work. AI-assisted coding tools (Anthropic's Claude, including via Claude Code) supported development of the `kiji-inspector` codebase, where they were used to draft boilerplate code, scaffold tests, and suggest refactors. The same class of tools assisted in preparing this manuscript, where they were used to improve language for clarity. All AI-assisted outputs, such as code, and texts were reviewed, executed, and revised by

the authors, who take full responsibility for the accuracy and integrity of the final content. Standard spell-checkers, grammar tools (e.g., Grammarly), and code linters (e.g., GitHub Co-Pilot) are excluded from this disclosure, consistent with the policy.

We distinguish this authoring assistance from the large language models that are the *subject* of our methodology. The generation model (Qwen3-VL-235B) and the subject model (Nemotron-3-Nano-30B) described in Methodology and Experimental Setup are components of the interpretability pipeline itself, not authoring aids, and their roles are documented in full in those sections. Consistent with the policy, no AI tool is listed as an author or contributor; authorship reflects human accountability alone.

This work was conducted as part of Dataiku's 575 Lab, the company's open source office. The source code for this project is available at https://github.com/dataiku/kiji-inspector.

Hannes Hapke and David Cardozo are employees of Dataiku Inc., and part of Dataiku's 575 Lab, the open source office. Carlos Huisa and Triveni Gandhi are affiliated with Dataiku Inc. Compute resources for this project have been provided by NVIDIA, Inc.

References

[1] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language Models Don't Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting." [Online]. Available: https://arxiv.org/abs/2305.04388

[2] T. Bricken *et al.*, "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning," *Transformer Circuits Thread*, 2023, [Online]. Available: https://transformer-circuits.pub/2023/monosemantic-features

[3] H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey, "Sparse Autoencoders Find Highly Interpretable Features in Language Models." [Online]. Available: https://arxiv.org/abs/2309.08600

[4] C. Juang, G. Paulo, J. Drori, and N. Belrose, "Open Source Automated Interpretability for Sparse Autoencoder Features." [Online]. Available: https://blog.eleuther.ai/autointerp/

[5] T. Gandhi, "Build Responsible GenAI Applications with the RAFT Framework." [Online]. Available: https://www.dataiku.com/stories/blog/build-responsible-genai-applications-with-the-raft-framework

[6] N. Elhage *et al.*, "Toy Models of Superposition," *Transformer Circuits Thread*, 2022, [Online]. Available: https://transformer-circuits.pub/2022/toy_model/index.html

[7] S. Rajamanoharan *et al.*, "Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders." [Online]. Available: https://arxiv.org/abs/2407.14435

[8] C. Olah, N. Cammarata, L. Schubert, G. Goh, M. Petrov, and S. Carter, "Zoom In: An Introduction to Circuits," *Distill*, 2020, doi: 10.23915/distill.00024.001.

[9] A. Conmy, A. N. Mavor-Parker, A. Lynch, S. Heimersheim, and A. Garriga-Alonso, "Towards Automated Circuit Discovery for Mechanistic Interpretability." [Online]. Available: https://arxiv.org/abs/2304.14997

[10] L. Sharkey, D. Braun, and B. Millidge, "Taking Features Out of Superposition with Sparse Autoencoders." [Online]. Available: https://www.alignmentforum.org/posts/z6QQJbtpkEAX3Aojj/interim-research-report-taking-features-out-of-superposition

[11] T. Schick *et al.*, "Toolformer: Language Models Can Teach Themselves to Use Tools." [Online]. Available: https://arxiv.org/abs/2302.04761

[12] S. Yao *et al.*, "ReAct: Synergizing Reasoning and Acting in Language Models." [Online]. Available: https://arxiv.org/abs/2210.03629

[13] S. Bills *et al.*, "Language Models Can Explain Neurons in Language Models." [Online]. Available: https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html

[14] A. M. Turner *et al.*, "Steering Language Models With Activation Engineering." [Online]. Available: https://arxiv.org/abs/2308.10248

[15] A. Zou *et al.*, "Representation Engineering: A Top-Down Approach to AI Transparency." [Online]. Available: https://arxiv.org/abs/2310.01405

[16] S. Rajamanoharan *et al.*, "Improving Dictionary Learning with Gated Sparse Autoencoders." [Online]. Available: https://arxiv.org/abs/2404.16014